

SOMMERSEMESTER 2023

Studiengang/Abschluss: Wirtschaftsinformatik / B.Sc.
Prüfungsfach: Programmierung
Prüfungsnummer: **5706**
Prüfer: Prof. Dr. Andreas Biesdorf
Anzahl der Arbeitsblätter: 12 Seiten (ohne Deckblätter)
Bearbeitungszeit: 90 Minuten

Mit Bleistift oder in roter Farbe geschriebene Ausführungen werden nicht gewertet!

Zugelassene Hilfsmittel:

Eine einseitig handbeschriebene DIN A4-Seite Python-Befehle oder -Syntax (z.B. die Syntax einer for- oder while-Schleife), jedoch keine Theorie oder Strukturen konkreter Implementierungen kompletter Aufgaben.

Matrikelnummer:

--	--	--	--	--	--

Datum der Klausur:

..... **Platznummer:**

Punkte: _____ von **90** möglichen

Note: _____

Handzeichen des Prüfers: _____

Hinweise zu Klausuren im FB Wirtschaft

- Mit Aufnahme der Prüfung bringen Sie zum Ausdruck, dass Sie sich für prüfungstauglich halten. Ein Rücktritt von der Klausur während der Bearbeitungszeit ist damit nur noch in ganz besonderen Ausnahmefällen möglich.
- Sollten Sie zu Beginn der Bearbeitungszeit prüfungstauglich sein, dann aber während der Bearbeitungszeit prüfungsuntauglich werden, müssen Sie die Klausur abbrechen, Ihre Prüfungsuntauglichkeit den Aufsichtführenden anzeigen und schnellstmöglich, in jedem Fall aber fristgerecht für ein ärztliches Attest sorgen.
- Mit der Abgabe Ihrer Klausur bringen Sie zum Ausdruck, dass Sie sich während der gesamten Bearbeitungszeit für prüfungstauglich gehalten haben. Ein Rücktritt ist dann nicht mehr möglich.
- Jacken und Taschen sind in ausreichender Entfernung vom Platz zu deponieren.
- Die Bearbeitungszeit beginnt erst, wenn alle Klausuren ausgeteilt sind.
- Es sind nur die zugelassenen Hilfsmittel zu verwenden.
- Das Verlassen des Raumes für Toilettengänge während der Klausur ist nur nach Meldung und Bestätigung der Aufsicht gestattet. Die vorübergehende Abwesenheit wird im Protokoll vermerkt.
- Es sind keine anderen als die ausgegebenen Bearbeitungsblätter zu benutzen.
- Auf dem Tisch sind nur Schreibutensilien erlaubt, keine Mäppchen.
- Jeder Teilnehmer muss sich mit einem Studierendenausweis oder Lichtbildausweis ausweisen.
- Geräusche und Störungen jeder Art sind zu vermeiden.
- Die Klausurensätze sind grundsätzlich nicht zu öffnen. Wer den Klausurensatz öffnet, trägt dafür Sorge, dass die Blätter (innerhalb der Bearbeitungszeit) sortiert und neu geheftet werden. Das Risiko des Verlustes von einzelnen Blättern trägt der Studierende!
- Es ist nicht erlaubt, Aufgabenstellungen auf gesondertes Papier abzuschreiben.
- Mobiltelefone, Smartphones und Smartwatches sind nicht erlaubt!
- Alle Studierenden bleiben auf ihren Plätzen und verhalten sich ruhig, bis die Klausuren vollständig eingesammelt sind. Folgen Sie den Anweisungen der Aufsichtführenden.
- Bei Täuschung oder versuchter Täuschung wird mit „nicht ausreichend“ bewertet.

Die Klausur besteht aus 9 Aufgaben zu den in der Lehrveranstaltung behandelten Themenbereichen. Die Punktzahl ist bei jeder Aufgabe mit angegeben.

Aufgabe	Mögliche Punkte	Erreichte Punkte
A1	7	
A2	8	
A3	5	
A4	10	
A5	10	
A6	10	
A7	20	
A8	15	
A9	5	
Summe	90	
Evtl. Bonus		

Bitte dokumentieren Sie die Antworten zu allen Aufgaben in der entsprechenden Textbox auf dem Blatt oder der entsprechend benannten Datei (z.B. aufgabe1.txt oder aufgabe7.py) in dem Ordner auf der VM.

Aufgabe 1

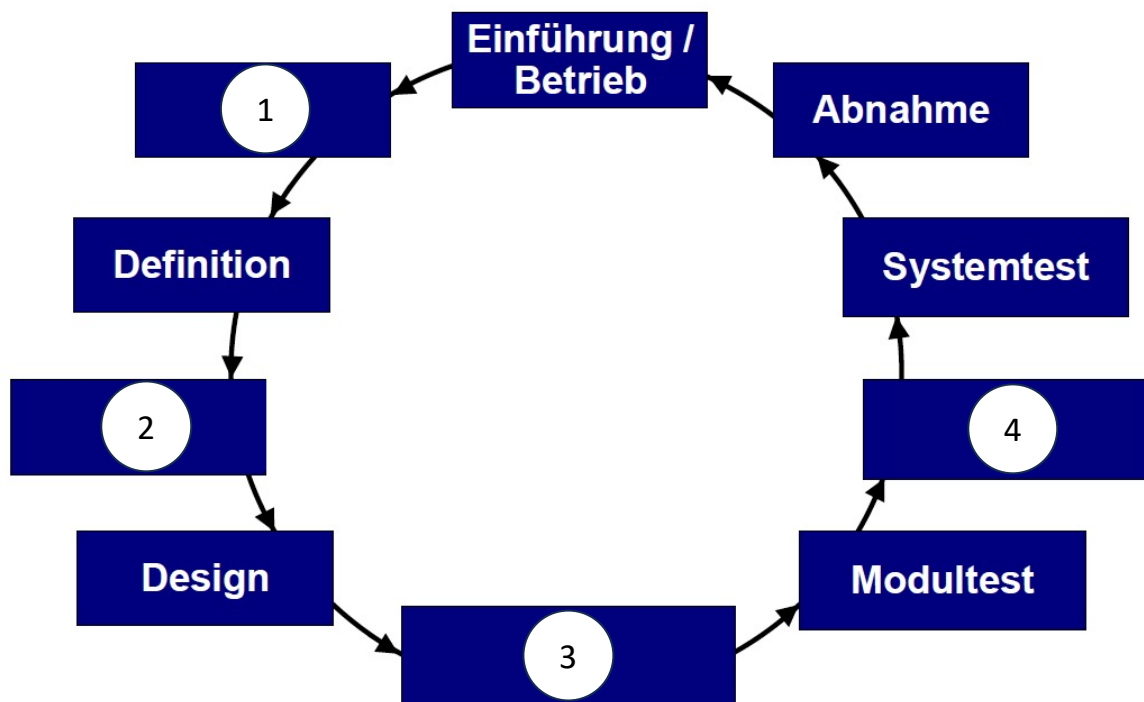
- 1) Was ist der Unterschied zwischen Programmierung und Software-Engineering? Erläutern Sie den Unterschied *stichpunktartig* in ihren Worten. [2 Punkte]

- 2) Was versteht man unter dem „Magischen Dreieck“ im Software-Engineering? Erläutern Sie das Konzept *stichpunktartig* anhand eines konkreten Beispiels. [3 Punkte]

- 3) Welche der folgenden Aussagen beschreibt am besten den Unterschied zwischen funktionalen und nicht-funktionalen Anforderungen in der Softwareentwicklung? Bitte kreuzen Sie die richtige Antwort an. [2 Punkte]
- ☐ Funktionale Anforderungen beziehen sich auf die Sicherheit und Leistung eines Systems, während nicht-funktionale Anforderungen sich auf die spezifischen Funktionen und Aufgaben beziehen, die ein System erfüllen soll.
 - ☐ Funktionale Anforderungen beschreiben, was ein System tun soll, während nicht-funktionale Anforderungen beschreiben, wie gut das System seine Funktionen ausführen soll.
 - ☐ Funktionale Anforderungen beziehen sich auf die Nutzererfahrung und das Aussehen eines Systems, während nicht-funktionale Anforderungen sich auf die internen Abläufe und Algorithmen beziehen.
 - ☐ Funktionale Anforderungen werden während der Systemwartung behandelt, während nicht-funktionale Anforderungen während der Systementwicklung berücksichtigt werden.

Aufgabe 2

Ergänzen Sie die vier fehlenden Begrifflichkeiten in folgendem Bild. Erläutern Sie stichpunktartig, welche Aufgaben in der jeweiligen Phase bearbeitet werden müssen und was das Ergebnis der Phase ist. [8 Punkte]



Begriff 1:

Begriff 2:

Begriff 3:

Begriff 4:

Aufgabe 3

- 1) Was ist der grundlegende Unterschied zwischen dem Wasserfallmodell und dem V-Modell? Erläutern Sie den Unterschied stichpunktartig. [2 Punkte]

- 2) Welches Phasenmodell sollte für die folgenden Projekte gewählt werden, wenn Sie ausschließlich zwischen dem Wasserfall- und dem V-Modell wählen können? Bitte ignorieren Sie, dass in der Realität sehr wahrscheinlich andere Phasenmodelle diskutiert und zum Einsatz kommen würden. Erläutern Sie die Empfehlung mit einem Satz. [3 Punkte]

Projektbeschreibung	Wasserfall- oder V-Modell?
Entwicklung eines einfachen mobilen Spiels für ein kleines Startup-Unternehmen. Die Anforderungen sind gut definiert und es wird erwartet, dass sich während des Entwicklungsprozesses keine wesentlichen Änderungen ergeben.	
Entwicklung eines kritischen Flugsicherheitssystems für eine internationale Fluggesellschaft. Das System muss höchste Sicherheitsstandards erfüllen und ausgiebig getestet werden. Änderungen in den Anforderungen sind während des Projekts wahrscheinlich.	

Projektbeschreibung	Wasserfall- oder V-Modell?
Entwicklung einer medizinischen Softwarelösung für ein Krankenhaus. Das System muss höchste Genauigkeit und Zuverlässigkeit gewährleisten. Es ist wichtig, dass während der Entwicklung eng mit dem medizinischen Fachpersonal zusammengearbeitet wird, um die Anforderungen zu verstehen und zu validieren.	
Entwicklung eines neuen Betriebssystems für Mobiltelefone mit innovativen Funktionen und einer komplexen Architektur. Das Projektteam besteht aus verschiedenen Abteilungen und Experten, die in enger Zusammenarbeit an der Entwicklung des Systems arbeiten.	
Entwicklung eines Druckers für ein Technologieunternehmen. Die Anforderungen sind gut definiert und es werden keine wesentlichen Änderungen während des Entwicklungsprozesses erwartet. Das Projektteam besteht aus einem kleinen Team erfahrener Ingenieure.	

Aufgabe 4

- a) Gegeben ist eine Python-Funktion, die den Body-Mass-Index (BMI) einer Person in Abhängigkeit von Gewicht (in kg) und Größe (in m) bestimmt:

prog_ss23/src/aufgabe04.py

```
def bmi_klassifizierung(gewicht, groesse):  
  
    bmi = gewicht / (groesse**2)  
  
    if bmi < 18.5:  
        return "Untergewicht"  
    elif bmi < 25:  
        return "Normalgewicht"  
    elif bmi < 30:  
        return "Übergewicht"  
    else:  
        return "Adipositas"
```

1. Benennen Sie fünf (möglichst überschneidungsfreie) Äquivalenzklassen zum Testen der Funktion. Erläutern Sie kurz, warum Sie diese Klassen gewählt haben. **[2.5 Punkte]**

2. Erstellen Sie in der Datei prog_ss23/test/test_aufgabe04.py Tests mithilfe von py-test, die die Funktion mit jeweils einem repräsentativen Wert aus jeder Äquivalenzklasse testet. **[2.5 Punkte]**

- b) Gegeben ist folgende iterative Python-Funktion, die eine Liste von Zahlen entgegennimmt und die Summe aller positiven Zahlen der Liste erstellt.

prog_ss23/src/aufgabe04.py

```
def addiere_positive_zahlen(zahlenliste):  
    summe = 0  
    for zahl in zahlenliste:  
        if zahl > 0:  
            summe += zahl  
    return summe
```

1. Benennen Sie fünf (möglichst überschneidungsfreie) Äquivalenzklassen zum Testen der Funktion. Erläutern Sie kurz, warum Sie diese Klassen gewählt haben. **[2.5 Punkte]**

2. Erstellen Sie in der Datei prog_ss23/test/test_aufgabe04.py Tests mithilfe von pytest, die die Funktion mit jeweils einem repräsentativen Wert aus jeder Äquivalenzklasse testet. **[2.5 Punkte]**

Aufgabe 5

Gegeben sei die folgende – offensichtlich nicht sehr robust implementierte – Funktion namens `string_operations`, die eine Liste von Strings und einen Index als Parameter entgegennimmt. Die Funktion soll den String am gegebenen Index der Liste zurückgeben, nachdem alle Leerzeichen entfernt und alle Buchstaben in Kleinbuchstaben umgewandelt wurden. **(10 Punkte)**

prog_ss23/src/aufgabe05.py

```
def string_operations(string_list, index):  
    if not isinstance(string_list, list) or not isinstance(index, int):  
        raise TypeError  
  
    return string_list[index].replace(" ", "").lower()  
  
# Beispielaufruf und Überprüfung  
assert string_operations(["Hallo Welt", "Python Exceptions"], 0) == "hallowelt"
```

Bitte überarbeiten Sie die **Funktion** `string_operations` um folgendes Verhalten:

- Sofern der erste Parameter keine Liste oder der zweite Parameter kein Integer ist, wird aktuell intern eine Exception vom Typ **TypeError** erzeugt. Bitte fangen Sie die Exception ab und geben eine benutzerdefinierte Fehlermeldung aus.
- Ergänzen Sie die Funktion um die Behandlung von Exceptions vom Typ **IndexError**, wenn der angegebene Index außerhalb des Bereichs der Liste liegt.
- Nur wenn **keine Exception** auftritt, geben Sie den modifizierten String zurück.
- Beenden Sie das Programm immer mit einer Nachricht, die bestätigt, dass die Funktion `string_operations` ausgeführt wurde, unabhängig davon, ob eine Exception aufgetreten ist oder nicht.

Aufgabe 6

Schreiben Sie ein Programm, das eine Textdatei liest, den Inhalt Zeile für Zeile mittels einer Caesar-Verschiebung verschlüsselt bzw. wieder entschlüsselt und das Ergebnis in eine Ausgabedatei schreibt. Alle nötigen Informationen und Befehle sollten über die Kommandozeile angegeben werden. **[10 Punkte]**

Hinweis: der Code für die Ver- und Entschlüsselung ist schon in der Datei vorgegeben und muss nicht implementiert werden!

Beispielhafter Aufruf:

```
python3 aufgabe06.py -i <infile> -o <outfile> -s <shift> -c/-d
```

Argumente / Flags auf der Kommandozeile:

- infile/-i: gibt die Eingabedatei an
- outfile/-o: gibt die Ausgabedatei an
- chiffrieren/-c: Eingabedatei soll verschlüsselt werden und in Ausgabedatei gespeichert werden
- dechiffrieren/-d: Eingabedatei ist schon verschlüsselt und soll dechiffriert und dann in Ausgabedatei gespeichert werden
- shift/-s: Shift um Anzahl Zeichen im Alphabet
- help/-h: Ausgabe des Programmaufrufs

Aufgabe 7

Gegeben ist der folgende Code einer *einfach* verketteten Liste. Die verkettete Liste hat lediglich eine Methode zum Anfügen eines neuen Elements sowie eine Methode zum Ausgeben der Inhalte.

prog_ss23/src/aufgabe07.py

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        if not self.head:
            self.head = Node(data)
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = Node(data)

    def print_list(self):
        current = self.head
        while current:
            print(current.data)
            current = current.next
```

- Bitte implementieren Sie eine Funktion `get(i)`, welche Ihnen das *i*-te Element der verketteten Liste zurückgibt **(4 Punkte)**
- Ergänzen Sie die Funktion „`insertAfterElement`“, die ein neues Element nach einem schon in der Liste enthaltenem Element hinzufügt **(6 Punkte)**
- Bitte erweitern Sie die bestehende Implementierung der Funktionen der einfach verketteten Liste auf eine *doppelt* verkettete Liste. **(10 Punkte)**
Hinweis: es ist nicht erforderlich eine Funktion zum Löschen von Elementen zu ergänzen.

Aufgabe 8

Die Buchstaben eines beliebigen Strings sollen alphabetisch sortiert werden.

Gegeben sei hierfür folgender Rahmen des Codes:

prog_ss23/src/aufgabe08.py

```
def quicksort(input_text):  
    # Ihre Implementierung hier  
    pass  
  
# Testen Sie Ihre Implementierung mit dem folgenden Code  
text = input("Bitte Text ohne Sonder- und Leerzeichen eingeben: ").lower()  
  
# Erwartete Ausgabe: identische Zeichen, jedoch alphabetisch sortiert  
print(quicksort(text))
```

Bitte implementieren Sie die Funktion quicksort unter Verwendung des Quicksort-Verfahrens. **[15 Punkte]**

Aufgabe 9

Gegeben ist eine Liste von Programmiersprachen und eine Reihe von Programmieraufgaben bzw. -anforderungen. Ordnen Sie jeder Aufgabe bzw. Anforderung die geeignetste Programmiersprache aus der Liste zu. Beachten Sie, dass einige Aufgaben mehrere geeignete Sprachen haben könnten, aber versuchen Sie, die beste Sprache basierend auf der üblichen Verwendung und den Stärken der jeweiligen Sprache zu wählen.

Bitte verbinden Sie: **5 Punkte**

Entwicklung einer iOS-App	☐	☐ WordPress
Erstellung einer Website mit interaktiven Funktionen	☐	☐ Erlang
Entwicklung einer Android-App	☐	☐ Golang
Schreiben von Skripten zur Automatisierung von Aufgaben auf einem Webserver	☐	☐ Java
Entwicklung eines massiv parallelen, numerischen Berechnungsprogramms	☐	☐ JavaScript
Erstellung eines Content-Management-Systems für einen Blog	☐	☐ Julia
Entwicklung eines Systems zur Verarbeitung und Verteilung von Echtzeit-Nachrichten	☐	☐ Kotlin
Schreiben einer Windows-Desktop-Anwendung	☐	☐ Ruby
Erstellung einer Webanwendung mit Backend-Server und Datenbankzugriff	☐	☐ Swift
Entwicklung eines Systems zur Handhabung von Server-Backends mit hoher Netzwerkleistung	☐	☐ Visual Basic